AI Utility Report — Test-Run Triage Assistant

QA Take-Home Assessment | Part 6 Task B (AI Utility Implementation)

Author: Uma Uka **Script:** `ai-utility/triage-report.mjs` **Date:** August 31, 2026

1. Use Case Chosen

The brief offered three example use cases. I built **"summarize a failed test run's logs into a triage note"** chosen specifically because it allowed using real data already in this repository (Part 2's own Playwright JSON reports, including actual `test.fail()` / `BUG-n` annotations) rather than a fabricated bug report or invented wallet-transfer edge cases.

The script, `ai-utility/triage-report.mjs`, reads a Playwright JSON report and produces a Markdown triage note that cleanly separates:

- **Genuinely new/unexpected failures:** Requires human attention immediately.
- **Tracked bugs that unexpectedly passed:** May be fixed — verify and promote out of `test.fail()`.
- **Known tracked bugs:** Still behaving as documented (no action needed).
- **Flaky tests:** Identified and flagged for separate investigation.

2. Prompt Design & Tool Schema Architecture

Key Design Principle: Categorization is done deterministically in code from the report's own status and annotation fields — the LLM never counts or classifies. Its sole responsibility is writing a concise human-readable summary and suggesting a likely root-cause area and priority for already-identified genuine failures. This directly applies the findings from the Part 6 Task A Critique: *LLMs excel at qualitative judgment but are unreliable at exact arithmetic and bookkeeping.*

Output is enforced into a fixed schema using strict tool choice (`strict: true` on the tool definition) rather than parsing free-text JSON. This guarantees schema-valid output directly from the API.

System Prompt (Verbatim):

```
You are a QA triage assistant. You will receive a JSON summary of one Playwright test run.
Known, already-tracked bugs and possibly-fixed bugs are provided for context only — they are
already handled by the caller and need no analysis from you. Your only job is: for each entry
in unexpected_failures, suggest a short likely_area, a one-sentence likely_cause hypothesis
based on its error_message, and a suggested_priority. Also write a 2-4 sentence executive_summary
of overall run health. You MUST return exactly one entry in unexpected_failures for every entry
given in the input, matching test_title and file exactly — never invent, merge, or omit an entry.
```

Tool Schema (Abridged):

```
{
  "name": "submit_triage_analysis",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": {
      "executive_summary": { "type": "string" },
      "unexpected_failures": {
        "type": "array",
        "items": {
          "properties": {
            "test_title": { "type": "string" },
            "file": { "type": "string" },
            "likely_area": { "type": "string" },
            "likely_cause": { "type": "string" },
            "suggested_priority": { "type": "string", "enum": ["P1", "P2", "P3"] }
          },
          "required": ["test_title", "file", "likely_area", "likely_cause",
"suggested_priority"],
          "additionalProperties": false
        }
      },
      "flaky_tests_note": { "type": "string" }
    },
    "required": ["executive_summary", "unexpected_failures", "flaky_tests_note"],
    "additionalProperties": false
  }
}
```

3. Example Input and Output

The script features a banner-labeled `--mock` mode that runs the entire pipeline end-to-end with a deterministic stand-in for the API call. Below is the input payload and actual generated Markdown output:

Input Payload Sample (`ai-utility/sample-reports/mixed-run.json`):

```
{
  "stats": { "expected": 2, "skipped": 0, "unexpected": 2, "flaky": 1, "duration": 842.5 },
  "unexpected_failures": [
    { "test_title": "creates a token with valid credentials", "file": "auth.spec.js",
"error_message": "Error: connect ECONNREFUSED 127.0.0.1:3001" }
  ],
  "possible_fixed_bugs": [
    { "test_title": "deleting an already-deleted booking returns 404", "file": "crud.spec.js",
"bug": "BUG-2: DELETE on a missing booking returns 405 instead of 404" }
  ],
  "flaky": [
    { "test_title": "GET /booking filtered by checkin/checkout date range...", "file":
"filtering.spec.js" }
  ]
}
```

Output Generated by `node ai-utility/triage-report.mjs`:

```
# Test Run Triage Note
**Run summary:** 2 expected, 2 unexpected, 1 flaky, 0 skipped (0.88s)

## Summary
[MOCK] 1 unexpected failure(s) detected alongside 1 known tracked issue(s) behaving as
documented.

## Needs Human Attention Now
### creates a token with valid credentials
File: auth.spec.js
Error: Error: connect ECONNREFUSED 127.0.0.1:3001
Likely area: network/connectivity
Likely cause: [MOCK] Error message suggests the service under test was unreachable.
Suggested priority: P1

## Possibly Fixed — Verify & Promote Out of test.fail()
deleting an already-deleted booking returns 404 (crud.spec.js) now passes despite being tracked
as: BUG-2: DELETE on a missing booking returns 405 instead of 404

## Known, Tracked — No Action Needed
BUG-1: DELETE /booking/{id} returns 201 Created instead of 200/204

## Flaky — Needs Investigation
- GET /booking filtered by checkin/checkout date range... (filtering.spec.js)
[MOCK] Flaky tests present: investigate separately.
```

4. Validation Approach

Three layers of validation are applied in strict order before any output is trusted:

1. **Strict Tool-Use Schema:** The API guarantees the response matches the JSON schema exactly (correct types, required fields, no extraneous properties), eliminating malformed JSON.
2. **Zod Re-validation (Defense in Depth):** The response structure is re-verified in application code independently of the API's guarantee.
3. **Semantic Grounding Check:** Every `test_title` and `file` pair referenced by the model must exist verbatim in the deterministic ground-truth list. If the model hallucinates a test name or omits an entry, the check fails and output falls back to deterministic-only sections.

Verification Script Output (`node ai-utility/verify.mjs`):

```
--- Case 1: well-formed, grounded analysis ---
PASS schema validation accepts a well-formed analysis
PASS grounding check finds no errors on real data

--- Case 2: hallucinated test name (invented by model) ---
PASS schema validation still accepts it
PASS grounding check catches the count mismatch
PASS grounding check names the specific hallucinated test
Grounding errors reported:
  Count mismatch: model returned 2 unexpected failures, ground truth has 1.
  - Model referenced a test not present in input: "handles concurrent booking writes"

--- Case 3: dropped entry (model omits a real failure) ---
PASS grounding check catches a silently dropped failure
ALL CHECKS PASSED
```

5. Operational Guidelines Before Production Deployment

- **Human-in-the-Loop Triage:** The note is a triage assistant, not an auto-ticketing tool. Nothing is automatically created, closed, or reassigned.
- **Deliberate Bug Promotion:** The "Possibly Fixed" section prompts human verification; removing `test.fail()` remains a reviewed code change.
- **Graceful Fallback on Failure:** If validation (schema or grounding) fails, the AI-generated qualitative section is discarded entirely, and the note displays deterministic test counts exclusively.